SOC TEAM

Task # 4
Browser Activity and Phishing Detection in Wazuh

Submitted by: Usman Javaid
SOC Analyst Intern, ITSOLERA (Pvt.) Ltd.

Table of Contents

1. Introduction 3
2. Objectives 3
3. Tools and Environment 3
4. Methodology 4
4.1 Browser History Monitoring with osquery 4
4.2 Wiring osquery into the Wazuh Agent 4
4.3 Detection Rules for Browser Activity 5
4.4 Threat Intelligence List 6
4.5 Rule Validation with wazuh-logtest 6
4.6 Live Validation on the Manager 7
4.7 Phishing Email Detection 8
4.8 Dashboard Verification 8
5. Findings 9
6. Challenges 10
7. Conclusion 11

1. Introduction

This report documents the completion of Task 4 assigned under the SOC Analyst Internship at ITSOLERA (Pvt.) Ltd. The task required configuring browser activity monitoring in Wazuh, implementing phishing email detection, and generating alerts for suspicious URLs, domains, and other phishing indicators. The work involved building two related but independent detection pipelines within the same Wazuh environment: one that monitors Chrome browsing history for signs of phishing, and one that scans email content for phishing indicators. Both pipelines feed into a shared detection objective, allowing alerts from either source to be reviewed together under a common phishing category.

The task was carried out on a lab environment consisting of a Wazuh manager running on a virtual machine and a Windows host acting as the monitored endpoint. The manager received data from the host through the Wazuh agent, and custom detection rules were written to identify phishing behaviour in both browser history and email content.

2. Objectives

- Configure Chrome browser history monitoring through Wazuh using osquery.
- Collect and forward browser history events from the Windows endpoint to the Wazuh manager.
- Implement phishing email detection rules within Wazuh.
- Generate alerts for suspicious URLs, domains, and other phishing indicators found in both browser activity and email content.
- Validate all detections through controlled test scenarios and confirm alerts appear correctly in the Wazuh dashboard.

3. Tools and Environment

- Wazuh Manager v4.14.5 (virtual machine, Linux)
- Wazuh Agent (Windows host)
- osquery (installed on the Windows host for Chrome history monitoring)
- Python 3 (used for the custom email parsing script)
- Google Chrome (source of browser history data)

- VirtualBox (host of the Wazuh manager virtual machine)
- OpenPhish community feed (source of real-world phishing domains used for the threat intelligence list)

4. Methodology

4.1 Browser History Monitoring with osquery

Wazuh does not natively monitor Chrome browsing history, so osquery was used to bridge this gap. osquery is a tool that allows the operating system to be queried like a database, and it was configured on the Windows host using its Automatic Table Construction (ATC) feature. This feature registers Chrome's History SQLite database as a queryable table without requiring the browser to be closed.

A scheduled query named chrome_history_query was configured to run every sixty seconds, selecting the URL, page title, visit count, and last visit time from Chrome's history. The configuration file below shows the ATC block pointing to the Chrome History file and the schedule block that runs the query on an interval.

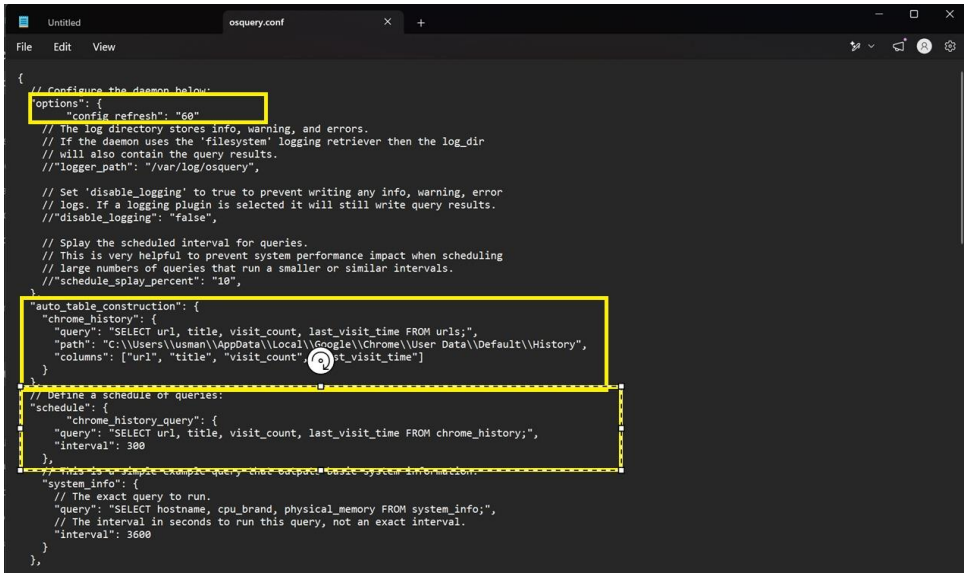


Figure 1: osquery.conf showing the auto_table_construction and schedule blocks for chrome_history_query

4.2 Wiring osquery into the Wazuh Agent

On the Windows agent, the osquery wodle module was enabled inside ossec.conf so that the Wazuh agent would follow the results file produced by osqueryd and forward new entries to the manager. Since osqueryd already runs as its own independent Windows service, the run_daemon

option was set to no so that Wazuh only tails the existing results log rather than starting a second instance of osquery.

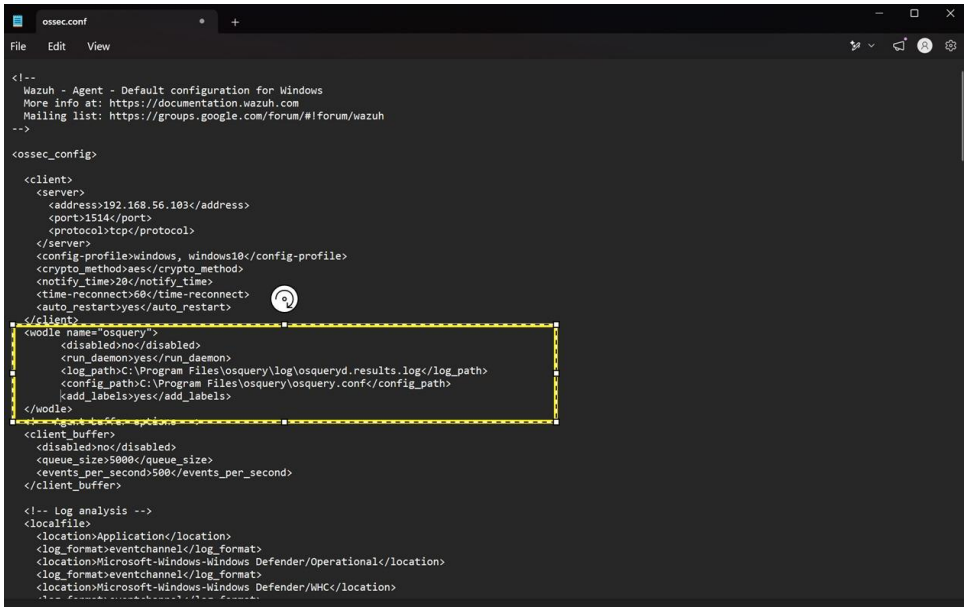


Figure 2: Agent-side ossec.conf showing the osquery wodle configuration

4.3 Detection Rules for Browser Activity

A set of custom rules was written on the Wazuh manager under the chrome_history and phishing rule group. The base rule matches any result coming from the chrome_history_query scheduled query, and the rules that follow check the visited URL against several phishing indicators, layered by confidence level:

- Suspicious keywords in the URL, such as login, verify, secure, or confirm.
- High risk top level domains commonly abused in phishing campaigns, such as .xyz, .top, .tk, and .click.
- A known phishing domain list sourced from the OpenPhish community feed.
- Free hosting and developer platforms frequently abused to host phishing pages, such as pages.dev, vercel.app, and blogspot.com.
- Brand impersonation patterns, where a well known brand name appears in a URL that does not belong to that brand's real domain.



```
<group name="chrome_history,phishing,">

  <!-- Match only results from our custom chrome_history_query -->
  <rule id="100220" level="3">
    <if_sid>24010</if_sid>
    <field name="osquery.name">chrome_history_query</field>
    <description>osquery: Chrome history query result - $(osquery.columns.url)</description>
    <options>no_full_log</options>
  </rule>

  <rule id="100221" level="10">
    <if_sid>100220</if_sid>
    <field name="osquery.columns.url" type="pcre2">(?i)(login|verify|secure|account-update|signin|confirm)</field>
    <description>osquery: Suspicious keyword in visited URL - possible phishing: $(osquery.columns.url)</description>
    <mitre>
      <id>T1566.002</id>
    </mitre>
    <group>phishing,</group>
  </rule>

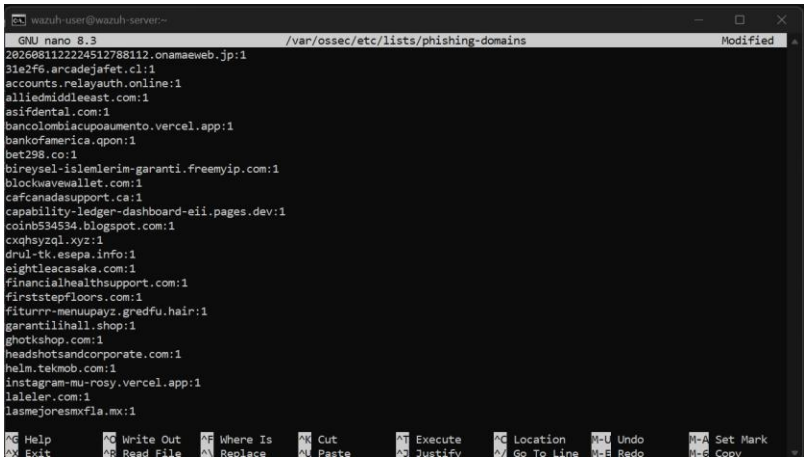
  <rule id="100222" level="12">
    <if_sid>100220</if_sid>
    <field name="osquery.columns.url" type="pcre2">(?i)\.(xyz|top|tk|click|gq|work|zip)(/|$)</field>
    <description>osquery: Visit to high-risk TLD - possible phishing domain: $(osquery.columns.url)</description>
    <mitre>
      <id>T1566.002</id>
    </mitre>
    <group>phishing,</group>
```

Figure 3: Custom rules 100220 to 100225 under the chrome_history, phishing rule group

Each rule carries a MITRE ATT&CK mapping to technique T1566.002, Spearphishing Link, so that alerts are automatically tagged with the correct tactic and technique for reporting purposes.

4.4 Threat Intelligence List

A list of real phishing domains was pulled from the OpenPhish community feed to give the detection rules a genuine, current threat intelligence source rather than fabricated test data. The domains were extracted, deduplicated, and saved as a reference list on the manager. During testing it was found that Wazuh's CDB list lookup performs an exact string match rather than a substring match, which does not work well against full URLs that include paths and query strings. Because of this, the working detection for known phishing domains was implemented as a regular expression rule rather than a CDB list lookup, while the CDB list itself was retained as the documented source of the domains used.



```
wazuh-user@wazuh-server:~
 GNU nano 8.3                    /var/ossec/etc/lists/phishing-domains                    Modified
20260811222245127881l2.onamaeweb.jp:1
31e2f6.arcadejafet.cl:1
accounts.relayauth.online:1
alliedmiddleeast.com:1
asifdental.com:1
bancolombiacupoaumento.vercel.app:1
bankofamerica.qpon:1
bet298.co:1
bireysel-islemlerim-garanti.freemyip.com:1
blockwavewallet.com:1
cafcanadasupport.ca:1
capability-ledger-dashboard-eii.pages.dev:1
coinb534534.blogspot.com:1
cxqhsyzql.xyz:1
drul-tk.esepa.info:1
eightleacasaka.com:1
financialhealthsupport.com:1
firststepfloors.com:1
fiturrr-menuupayz.gredfu.hair:1
garantilihall.shop:1
ghotkshop.com:1
headshotsandcorporate.com:1
helm.tekmob.com:1
instagram-mu-rosy.vercel.app:1
laleler.com:1
lasmejoresmxfla.mx:1

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

Figure 4: Phishing domain list sourced from the OpenPhish feed

4.5 Rule Validation with wazuh-logtest

Before relying on live traffic, each rule was validated individually using the wazuh-logtest utility. A sample log matching the format produced by the osquery wodle was fed into the tool, and the resulting rule match, severity level, and MITRE mapping were confirmed. The example below shows a URL matching a known phishing domain from the threat intelligence list, correctly triggering rule 100223 at severity level 14.



Figure 5: wazuh-logtest output confirming rule 100223 fires correctly on a known phishing domain

4.6 Live Validation on the Manager

Once the rules were confirmed through wazuh-logtest, the full pipeline was tested end to end using live Chrome browsing activity. The alerts log on the manager was monitored in real time while the Windows host generated genuine browsing activity, confirming that events were being received, decoded, and matched against rule 24010, the base osquery result rule, and the custom phishing rules that build on it.

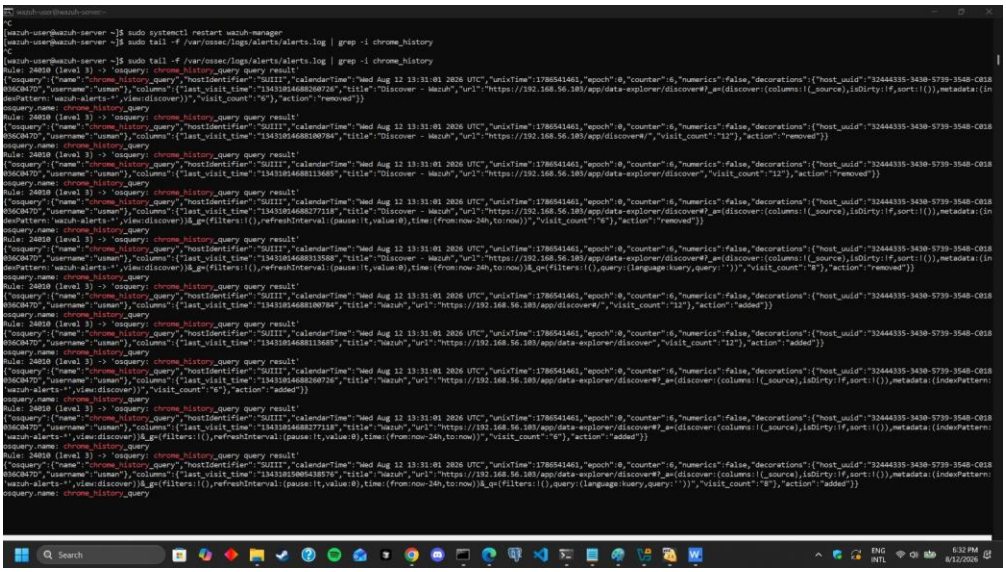


Figure 6: Live alerts log showing chrome_history_query results being received and processed by the manager

4.7 Phishing Email Detection

The second half of the task required detecting phishing indicators in email content, which is a separate data source from browser history and needed its own pipeline. Since a live mail server was not available, a Python script was written to parse .eml files and extract the fields relevant to phishing detection: sender address and domain, reply-to address and domain, subject line, links found in the email body, and any attachments.

The script specifically flags a mismatch between the sender's domain and the reply-to domain, which is a common indicator used in phishing and business email compromise attempts, and it flags attachments with suspicious or double extensions, such as a file appearing to be a PDF but actually ending in .exe. The script outputs one JSON line per email into a log file that Wazuh reads through a localfile block on the agent, in the same way any other monitored log is read.

On the manager side, six rules were written under the phishing_email and phishing rule group, covering reply-to domain mismatches, urgency based subject lines, suspicious attachments, high risk sender domains, known phishing domains referenced in the email body, and brand impersonation in the sender's display name. Each of these was tested individually using wazuh-logtest with sample emails covering both clean and phishing scenarios before being validated on live test data.

4.8 Dashboard Verification

With both pipelines confirmed working, the Wazuh dashboard was used to filter and review alerts by rule group. Filtering by rule.groups: chrome_history surfaced alerts generated from real Chrome browsing activity, and filtering by rule.groups: phishing_email surfaced alerts generated from the email parsing pipeline. Because both rule groups also share the common phishing tag, a single filter on rule.groups: phishing brings together alerts from both sources, which reflects the overall goal of the task: generating alerts for suspicious URLs, domains, and phishing indicators regardless of where they originate.

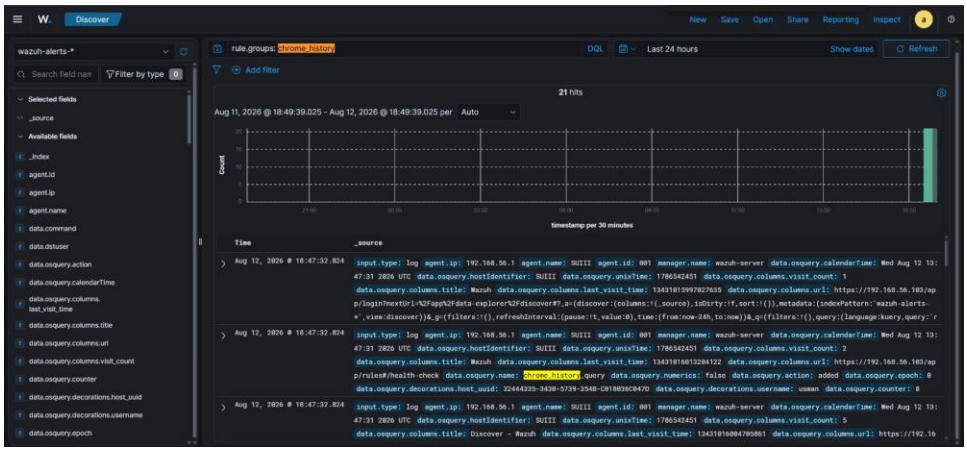


Figure 7: Wazuh dashboard filtered on rule.groups: chrome_history, showing twenty one alerts generated from live browsing activity

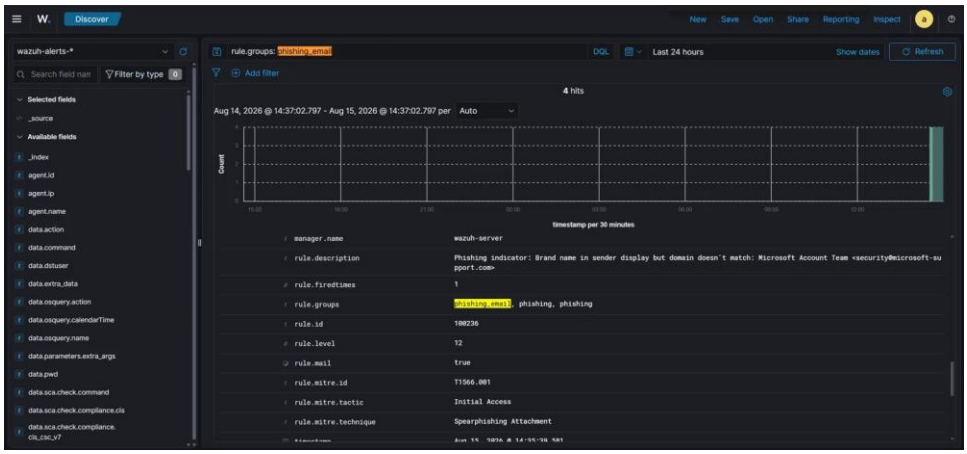


Figure 8: Wazuh dashboard filtered on rule.groups: phishing_email, showing a brand impersonation alert (rule 100236) with correct MITRE mapping

5. Findings

- Chrome browsing history can be reliably monitored through Wazuh by combining osquery's Automatic Table Construction feature with a scheduled query, without needing to close the browser.
- Wazuh's built in osquery integration wraps forwarded results in an outer osquery object, which is a detail that must be accounted for when writing rule field references, since it differs from how a manually pasted test log is structured.
- CDB list lookups in Wazuh perform exact matching, which limits their usefulness for free text fields like full URLs. Regular expression based rules were a more practical solution for domain matching in this case.

- Layering multiple detection rules of increasing severity, from a general keyword match up to a known domain match, gives a more realistic picture of phishing risk than relying on a single rule.
- Email based phishing detection requires a separate pipeline from browser based detection, since the underlying data (sender, subject, attachments) is structurally different from a visited URL, even though both ultimately feed into the same phishing alerting objective.
- A shared rule group naming convention across both pipelines (phishing) allows alerts from unrelated data sources to be reviewed together in a single dashboard filter.

6. Challenges

Several practical challenges were encountered while completing this task, most of which were resolved through direct troubleshooting on the manager and the agent.

- The Wazuh manager's virtual disk filled up completely during testing, largely due to the logall setting being enabled and archiving every event rather than only alerts. This caused the OpenSearch indexer to stop accepting new data and eventually caused the manager service to fail. The disk was resized, the archive logs were cleared, and logall was disabled to prevent the issue from recurring.
- An attempt to enable EFI and Secure Boot on the virtual machine, intended to resolve an earlier boot issue, instead broke the boot process entirely, since the underlying OVA image was built for legacy BIOS boot. This was resolved by disabling EFI again.
- The disk fill up event and the resulting unclean shutdowns caused filesystem level corruption on the manager's virtual disk, which was repaired using xfs_repair, and one corrupted OpenSearch index had to be removed and allowed to rebuild.
- Given the extent of the corruption, the manager was ultimately rebuilt from a fresh Wazuh OVA. All custom rules, the threat intelligence list, and the email parsing script had already been backed up, so the rebuild only required reapplying existing configuration rather than redoing the detection logic from scratch.
- Field naming inconsistencies between manually constructed test logs and real wodle generated traffic initially caused rules to appear correct in testing but not fire on live data. This was resolved by confirming the exact field structure of live alerts and adjusting the rule definitions accordingly.

These issues, while time consuming, provided practical experience in Wazuh infrastructure troubleshooting, including service dependency chains, disk space management, and filesystem recovery, which are directly relevant to real world SOC operations.

7. Conclusion

Task 4 was completed successfully. Chrome browser history monitoring was implemented using osquery and integrated into Wazuh through the osquery wodle, with six custom rules detecting suspicious keywords, high risk domains, known phishing domains, abused hosting platforms, and brand impersonation in visited URLs. A separate phishing email detection pipeline was built using a Python parsing script and six additional custom rules covering sender and reply-to mismatches, urgency based language, suspicious attachments, and brand impersonation in email content.

Both pipelines were validated first through wazuh-logtest and then confirmed working end to end on live data, with alerts visible and correctly categorised in the Wazuh dashboard. The task also involved resolving several infrastructure issues on the Wazuh manager, which reinforced practical troubleshooting skills alongside the core detection engineering objective of the task.